

Développement et intégration de chatbots multi-usages

Interface client et interface staff

Stage de fin d'étude



Tuteur entreprise: Laura DEGIOANNI

Référent Stage: Jean MARTINET

ACHIR Charaf

EIT Digital DS

2024/2025

Remerciements

Je remercie chaleureusement l'ensemble de l'équipe Innovation pour leur accueil, leur bienveillance et leur accompagnement tout au long de ces six mois. Un remerciement particulier à Laura Degioanni, ma cheffe d'équipe et tutrice de stage, pour sa disponibilité, sa confiance et son soutien au quotidien.

J'exprime également toute ma gratitude à Mr Jean Martinet et Mme Françoise Baude, mes enseignants référents à Polytech Nice Sophia, pour leur accompagnement constant, leurs conseils avisés et leur disponibilité tout au long de mes deux années de master au sein du programme EIT Digital. Leur soutien a contribué de manière significative à mon développement académique, professionnel, mais aussi personnel.

Enfin, je tiens à remercier mes parents, ma sœur et mes proches pour leur encouragement et leur présence indéfectible, qui m'ont permis de mener à bien cette expérience enrichissante.

Sommaire

Remerciements	2
Sommaire	3
Résumé	4
Mots clés	5
Introduction	6
PARTIE 1. Contexte de réalisation du stage	7
1.1. SaaSOffice et le GROUPE COURTIN	7
a) Structure et chiffres-clé	7
b) Leviers de compétitivité	8
1.2. L'équipe Innovation: une équipe dédiée à l'intelligence artificielle	9
a) Présentation de l'équipe Innovation	9
b) Les projets en cours	9
1.3. Politique DDRS de l'entreprise	10
PARTIE 2. Synthèse des travaux réalisés	12
2.1. Rappel de ma mission	12
a) Objectifs visés de mon travail	12
b) Gains attendus pour SaaSOffice et l'équipe Innovation	12
2.2. Réalisations	13
a) Moyens mis à ma disposition	13
b) Actions entreprises seul et en collaboration	14
Exploration des frameworks de monitoring:	14
Choix des techniques d'évaluation:	15
Implémentation technique:	16
Multilingue:	21
Création de lead:	22
Sécurisation des données:	24
Système de reporting:	25
Système d'alerting:	26
Agent citation:	28
Contextual chunker:	29
Tool statistiques clients:	29
Liens hypertexte:	30
c) Résultats obtenus et gains réalisés pour l'équipe Innovation	32
PARTIE 3. Impressions personnelles et acquis	34
3.1. Mes acquis au regard de ma mission chez SaaSOffice	34
a) Acquis personnels techniques	34

b) Acquis personnels humains	34
3.2. Acquis personnels et professionnels	35
a) Découverte du métier de développeur spécialisé en IA	35
Conclusion	36
Glossaire	37
Sitographie	38
Bibliographie	38
Annexes	39

Résumé

Du 3 Mai au 29 Août 2025, j'ai eu la chance d'effectuer mon stage de 5ème au sein de l'équipe Innovation de SaaSOffice, une start-up qui développe un outil 360° de gestion centralisée des espaces de travail, à destination des opérateurs de flex office et de coworking, combinant un pilotage orienté ROI pour optimiser la rentabilité des mètres carrés et une application tout-en-un pour améliorer l'expérience quotidienne au bureau. Au cours de ce stage, j'ai participé au développement et au déploiement de deux chatbots visant des utilisateurs-cible différents: Les end-users, c'est-à-dire les visiteurs des sites des entreprises clientes de SaaSOffice, et les Front Office Managers (FOM) de ces entreprises.

Ce projet nécessite l'utilisation de plusieurs langages de programmation, principalement Python, utilisé pour une grande partie du back-end afin de faciliter l'utilisation des LLMs, puis du HTML, Javascript, et CSS pour le front-end. Des connaissances en conteneurisation via Docker et Kubernetes ont aussi été requises pour la mise en production des projets au cours du stage.

Pendant six mois, j'ai eu l'occasion de découvrir le métier d'ingénieur informatique dans le contexte d'une start-up, ainsi que de maîtriser par la pratique le mode opératoire Agile notamment le framework Scrum avec toutes ses cérémonies. Ce stage m'a aussi permis d'apprendre concrètement le processus de développement en entreprise, notamment pour des problématiques liées à l'Intelligence Artificielle.

Mots clés

IA	LLM	Full-stack
Chatbot	RAG	Scrum

Introduction

Les agents conversationnels, ou chatbots, sont devenus des outils incontournables dans de nombreux domaines, allant du commerce en ligne au support technique, en passant par l'assistance administrative. Grâce aux récents progrès de l'intelligence artificielle, et notamment à l'émergence des modèles de langage de grande taille (LLMs), ces systèmes sont désormais capables de tenir des conversations plus naturelles, de comprendre des demandes complexes, et d'interagir avec des systèmes d'information en temps réel.

Dans le secteur du coworking, en plein essor depuis plusieurs années, les opérateurs cherchent à améliorer à la fois l'expérience de leurs clients et l'efficacité de leur gestion interne. C'est dans ce contexte que la start-up SaaSOffice, spécialisée dans les logiciels de monitoring et de gestion de sites de coworking, développe des solutions innovantes intégrant entre autres l'intelligence artificielle. Pour répondre aux besoins variés de ses utilisateurs, j'ai participé, avec l'équipe Innovation, à l'intégration de deux chatbots distincts au sein de sa plateforme :

Le premier chatbot est destiné aux sites publics des opérateurs de coworking clients de la société. Il a pour objectif d'informer les visiteurs sur les espaces disponibles, de répondre à leurs questions courantes, et de générer des opportunités commerciales. Il s'inscrit dans la stratégie de marketing digital, en ouvrant un nouveau canal d'acquisition.

Le second chatbot, plus technique, est intégré directement au logiciel de gestion proposé par SaaSOffice. Il est conçu pour assister les Front Office Managers dans leurs tâches quotidiennes en répondant à des questions sur la documentation interne, sur l'utilisation de la plateforme, sur les contrats des clients finaux ou encore sur la disponibilité des espaces, tout en facilitant la formation de nouveaux entrants.

Très intéressé par les applications concrètes de l'IA et par les problématiques liées à l'interaction homme-machine, ce stage représentait une opportunité idéale pour approfondir mes compétences techniques tout en découvrant les enjeux liés au développement logiciel dans un environnement start-up.

Dans ce rapport, je présenterai dans un premier temps le contexte de mon stage et les outils utilisés, puis les spécificités des deux chatbots développés, avant de conclure par un retour sur les compétences acquises et les enseignements tirés de cette expérience.

PARTIE 1. Contexte de réalisation du stage

1.1. SaaSOffice et le GROUPE COURTIN

a) Structure et chiffres-clé

Le [GROUPE COURTIN](#) [1] est un acteur français diversifié, implanté principalement dans les secteurs de l'immobilier, de l'énergie verte, et des nouvelles technologies. Porté par une vision entrepreneuriale forte, le groupe investit dans des projets à la fois durables et innovants, avec une volonté claire de digitaliser les métiers traditionnels, notamment ceux liés à l'immobilier tertiaire.

Parmi les structures clés du groupe figure [Flex-O](#) [2], une entreprise spécialisée dans la gestion d'espaces de travail flexibles (flex offices), proposant des bureaux opérés haut de gamme à destination des entreprises. Dans une logique de montée en puissance technologique, Flex-O avait développé en interne plusieurs outils numériques pour piloter son activité. Ces outils, initialement conçus au sein de la DSI Flex-O, ont été à l'origine de la création d'une nouvelle entité indépendante : [SaaSOffice](#) [3].

Née du carve-out de la direction technique de Flex-O, SaaSOffice est aujourd'hui une start-up à part entière, intégrée au portefeuille technologique du GROUPE COURTIN. Elle a pour mission de développer des solutions logicielles B2B (en mode SaaS) destinées aux acteurs de l'immobilier tertiaire, en mettant l'accent sur l'automatisation, la gestion des flux commerciaux, la valorisation des données, et plus récemment, l'intelligence artificielle appliquée.

Depuis son inauguration en Décembre 2024, SaaSOffice a vu son nombre d'employés passer de seulement quatre personnes, à une cinquantaine de salariés à mon arrivée début mars 2025. Cette évolution rapide a nécessité de multiples changements d'organisation, certains ayant eu lieu juste avant mon arrivée en stage, d'autres pendant comme le rachat de la société [WelcomR](#) [4] qui a rajouté une dimension hardware au produit via l'installation de poignées connectées.

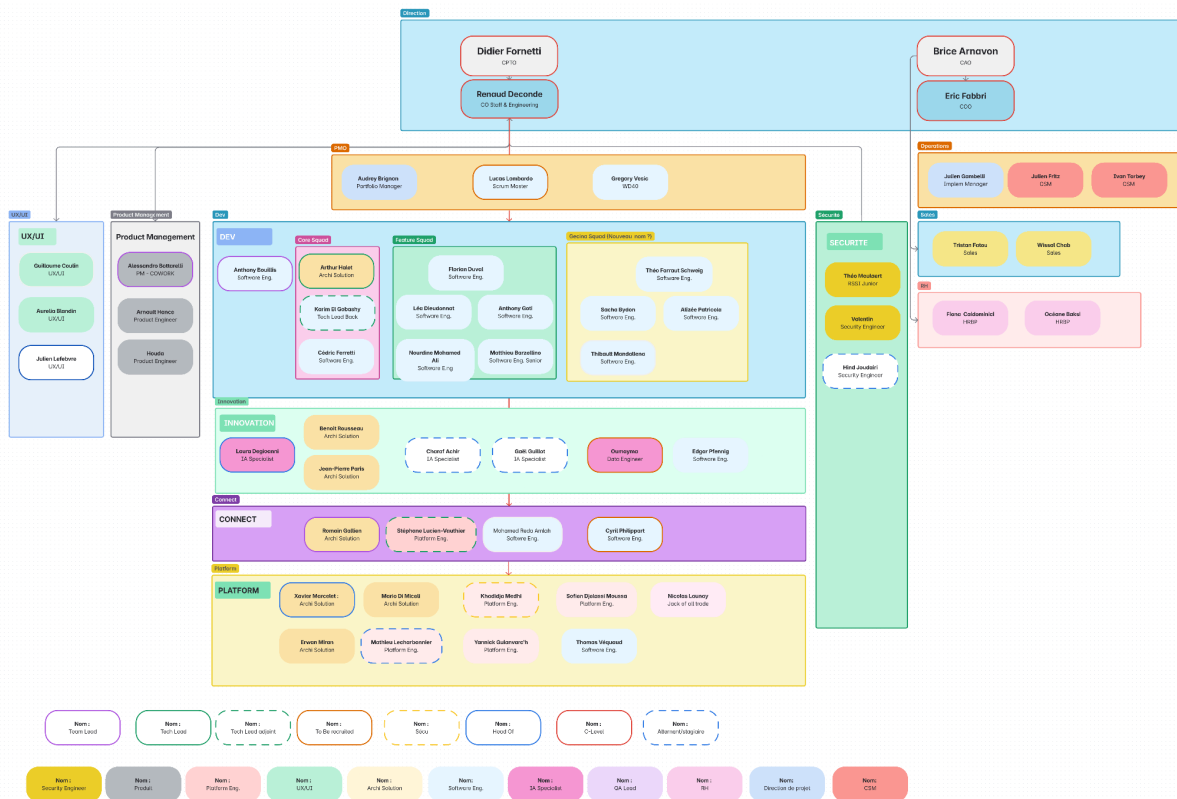


Fig. 1 - Structure SaaSOffice, Mai 2025

b) Leviers de compétitivité

Les problématiques majeures que les opérateurs de flexoffice et de coworking rencontrent sont principalement dues à la fragmentation des solutions logicielles. Ces opérateurs se retrouvent souvent à devoir gérer leur activité à l'aide de nombreux logiciels distincts, entraînant ainsi une augmentation des coûts, une complexité dans la gestion et un manque d'efficacité opérationnelle. Ce contexte démontre clairement le besoin pressant d'une solution unifiée et intégrée, qui simplifie et rationalise la gestion des espaces de coworking ; un besoin auquel SaaSOffice répond.

Par ailleurs, la modularité de la solution développée par SaaSOffice représente un levier de compétitivité important : les entreprises peuvent choisir d'activer uniquement les fonctionnalités dont elles ont réellement besoin, ce qui leur permet d'adapter le coût du produit à la taille de leur structure et à la maturité de leur activité. Ce modèle flexible, souvent associé à une tarification par module ou par usage, contribue à rendre la solution

accessible à un large éventail d'opérateurs, tout en favorisant leur montée en charge progressive.

1.2. L'équipe Innovation: une équipe dédiée à l'intelligence artificielle

a) Présentation de l'équipe Innovation

Mon stage s'est déroulé au sein de l'équipe Innovation, une équipe dédiée au développement de solutions impliquant l'IA. À mon arrivée, elle était composée de Laura Degioanni, AI Spécialist et responsable d'équipe, Benoît Rousseau et Jean-Pierre Paris, responsables de l'architecture des projets, et Edgar Pfennig, Software Engineer. À cette équipe s'est ajouté Gaël Guillot, stagiaire en quatrième année d'études, qui était présent durant 4 mois de mon stage d'avril jusqu'au mois de juillet, Oumayma Benyahya, Data Engineer qui nous a rejoint un deux mois après mon arrivée, et enfin le stagiaire Chidiebere Okoene, arrivé 1 mois avant mon départ.

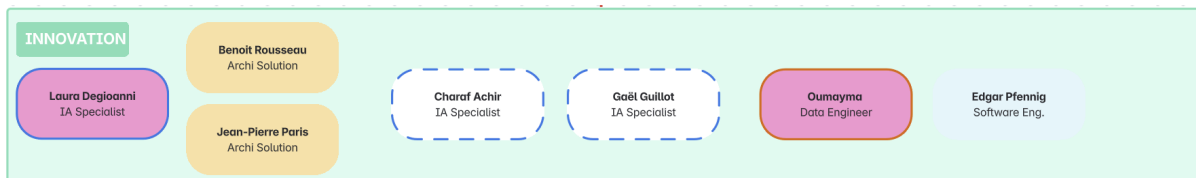


Fig. 2 - Équipe Innov, Août 2025

b) Les projets en cours

Les projets de développement de l'équipe Innovation actuellement en cours sont les suivants:

- **Chatbot-acquisition** : Un chatbot à déployer sur le site public du client historique Flex-O, servant à renseigner les visiteurs du site sur les services proposés par l'entreprise, et permettant la possibilité de recontacter l'utilisateur via un canal d'acquisition de prospects.
- **AI Assistant** : Un outil du Front Manager (FM), le logiciel de gestion fourni par SaaSOffice, permettant l'accès rapide aux différentes ressources disponibles sur l'application, afin de fluidifier la résolution de tâches simples et d'accélérer la formation des collaborateurs.

- **Génération de plans 2D** : Travail sur la digitalisation de plans issus de données hétérogènes en utilisant des librairies de computer vision (plans de différents formats, données brutes non nettoyées..)

1.3. Politique DDRS de l'entreprise

Dans son rapport annuel ESG (Environnement, Social et Gouvernance), SaaSOffice présente ses pratiques dans les trois secteurs:

- **Environnemental** : Le bâtiment Ecoryzon, dans lequel la start-up est installée, est éco-labellisé et conçu pour limiter le bilan carbone de l'entreprise, via l'utilisation de détecteurs de présence évitant le gaspillage énergétique pour l'éclairage, de poubelles de tri, de thermostat régulés, etc... Ces mesures ont permis d'obtenir la certification [BREEAM](#) [5] « Very good », et de diviser par trois la consommation énergétique du bâtiment. À cela s'ajoute une politique zéro-papier au sein de l'entreprise, la rédaction d'un bilan carbone GHG couplé à un plan d'action sur les années 2025 et 2026, ainsi que onze bornes de recharge pour les véhicules électriques actuellement en cours d'installation.
- **Social** : Sur le plan social, le SaaSOffice continue de promouvoir la qualité de vie au travail, en favorisant le télétravail pour les collaborateurs qui le souhaitent, tout en appliquant une politique de rémunération motivante, à l'aide de négociations annuelles obligatoires et de la participation des salariés à l'actionnariat de la société. Dans cette continuité et notamment grâce à sa politique RH moderne, SaaSOffice souhaite offrir à ses collaborateurs un cadre de travail bienveillant où l'initiative et les échanges sont encouragés. Un suivi de la répartition et de l'équité homme/femme, ainsi des parcours de carrière et plans de formation documentés, communiqués et alimentés par un canal spécifique pour recueillir les demandes de chaque collaborateur aident à lutter contre différentes formes de discrimination notamment à l'embauche.
- **Gouvernance** : Sur ce plan, SaaSOffice va déjà au-delà des préconisations et obligations réglementaires imposées aux sociétés répondant aux critères des PME au sens communautaire, notamment en harmonisant les CGV et CGU afin d'offrir une politique uniforme de gestion des données personnelles, répondant pleinement aux obligations du règlement européen, et d'intégrer ses engagements en matière de lutte contre la corruption et le blanchiment d'argent ainsi que les violations des droits

de l'homme.

Je suis en partie d'accord avec le rapport ESG. Les conditions de vie au travail m'ont paru excellentes, l'ambiance au sein des bureaux était majoritairement agréable et chaleureuse, et j'ai bénéficié d'un accompagnement continu en tant que stagiaire au long de ces six mois. En guise d'exemple qui démontre des pratiques de développement durable , dans le cadre de mon travail au sein de l'équipe Innov, les modèles mistral-small et mistral-medium étaient utilisés pour le chatbot-acquisition et l'AI Assistant respectivement, à la fois pour limiter l'envoi de données à l'international, et pour réduire le coût environnemental des appels.

PARTIE 2. Synthèse des travaux réalisés

2.1. Rappel de ma mission

a) Objectifs visés de mon travail

Mes deux objectifs principaux au cours de mon stage étaient les suivants: Premièrement, améliorer le fonctionnement d'un chatbot existant déployer sur le site public de l'entreprise Flex-O à destination des end-users, intitulé "*chatbot-acquisition*", ayant pour but de répondre aux questions des visiteurs du site sur les différents centres, tarifs et procédures de réservation, ainsi que d'effectuer automatiquement la création d'un prospect dans le module CRM de l'entreprise. Deuxièmement, développer et déployer un chatbot, l'*IA Assistant*, disponible depuis le Front Manager, c'est-à-dire l'interface du logiciel de gestion fourni par SaaSOffice. Celui-ci sert à renseigner les FOMs sur les capacités du logiciel, la documentation de leur entreprise, leurs contrats, leurs clients, ainsi que de nombreuses autres fonctionnalités. Enfin, un autre objectif majeur de mon stage consistait à déployer un framework de monitoring et de reporting dédié à ces chatbots, afin de suivre leurs performances, d'identifier les axes d'amélioration et de faciliter leur évolution continue.

b) Gains attendus pour SaaSOffice et l'équipe Innovation

Les gains attendus étaient l'avancement sur les deux chatbots afin d'ajouter de la valeur au produit fourni par SaaSOffice, ainsi que la documentation nécessaire pour assurer la passation des projets. Un troisième axe concernait la mise en place d'un framework de monitoring et de reporting, afin de doter l'équipe Innovation d'outils de suivi précis sur l'utilisation des chatbots, la qualité des réponses fournies et les taux de conversion générés. Ce dispositif visait également à faciliter la prise de décision grâce à des indicateurs clairs et exploitables.

Le chatbot acquisition avait pour objectif de remplacer l'ancienne solution de génération d'opportunités sur le site de Flex-O, qui reposait sur une simple fenêtre de chat *Respond.io*. Cette interface était construite autour de parcours figés, sans aucune capacité à comprendre le langage naturel, ce qui la rendait peu interactive et frustrante pour les utilisateurs.

L'analyse des conversations passées montrait clairement que les utilisateurs s'attendaient désormais à pouvoir s'exprimer librement, comme avec un véritable chatbot intelligent. Ce décalage entre leurs attentes et les capacités de l'outil se traduisait par un faible taux de conversion des leads. Grâce au monitoring et au reporting mis en place, il devenait possible de mesurer de manière continue ces indicateurs de satisfaction et d'optimiser progressivement les performances du chatbot.

Quant à l'AI Assistant, les améliorations attendues consistaient à récupérer rapidement et intuitivement les données du Front Manager, à fluidifier la navigation de son interface et à répondre aux questions des FOMs à l'aide des documents fournis par SaaSOffice et ses Clients (CGV, guide des standards, etc.), afin qu'une personne n'ayant pas reçu de formation extensive sur le logiciel ou ne disposant pas d'une compréhension approfondie des règles métier de l'entreprise cliente puisse néanmoins gérer efficacement les espaces de travail et contrats de celle-ci. Ici encore, le framework de monitoring permettait de suivre l'évolution des interactions et de s'assurer que les réponses fournies étaient pertinentes et réellement utiles aux utilisateurs finaux.

2.2. Réalisations

a) Moyens mis à ma disposition

Dès mon arrivée à SaaSOffice, j'ai eu accès à un MacBook Pro M4 (MacOS étant utilisé pour des raisons de sécurité et d'harmonisation des environnements de développement), un écran supplémentaire, et un clavier. Mais aussi un budget pour choisir un framework de monitoring payant. Mon travail étant celui de développeur, le reste des outils à ma disposition étaient majoritairement des licences logiciels, comme la suite Microsoft Office, ou bien l'accès aux tokens et api nécessaires à l'utilisation et au monitoring des LLMs (Mistral, OpenAI, Langchain, Langfuse).

Enfin, la disponibilité et l'encadrement fournis par les membres de l'équipe Innovation, ainsi que par les équipes Dev, Platform, UX/UI et Sécurité ont été d'une grande aide tout au cours du stage, non seulement dans la réalisation de mes tâches, mais aussi pour mon intégration

au sein de l'entreprise, via des points d'introduction avec les différentes équipes organisés par ma tutrice de stage et chef d'équipe, Laura.

b) Actions entreprises seul et en collaboration

Durant la majorité de mon stage, j'ai travaillé sur les projets chatbot-acquisition et ai-assistant en parallèle. Afin d'améliorer la clarté et la concision de ce rapport, je commencerai par présenter la partie monitoring commune aux deux projets, puis je détaillerai mes missions spécifiques sur chacun d'eux, plutôt que de suivre un ordre strictement chronologique.

Monitoring :

Après une semaine rapide de mise à niveau sur le fonctionnement de la start-up et la place de l'équipe Innovation en son sein, ma première « grosse » tâche a été de concevoir et d'implémenter un framework de monitoring pour les différents chatbots mis en production.

Pour mener à bien cette mission, je suis d'abord passé par une phase d'exploration et de collecte d'informations sur les solutions existantes d'observabilité et d'évaluation pour les modèles LLM et les pipelines RAG.

Exploration des frameworks de monitoring:

J'ai testé plusieurs approches : Langfuse, LangSmith, TruLens, Helicone, PromptLayer, ainsi qu'un montage maison basé sur OpenTelemetry + Grafana. Chaque outil apportait des avantages, mais pas toujours adaptés à nos contraintes.

- **LangSmith** s'est révélé excellent pour piloter des datasets et faire des campagnes d'évaluations/expériences, mais je ne l'ai pas retenu principalement pour des raisons de coût beaucoup trop élevé par rapport à d'autre framework disponible.
- **TruLens** propose un bon socle d'évaluations (notamment des critères de factualité et de "groundedness"), mais l'UI et l>alerting étaient moins matures pour un suivi quotidien multi-agents.
- **Helicone** est pratique en proxy d'appels LLM, mais impose un reroutage via leur endpoint (latence, risque de lock-in, et moins de granularité fine sur nos spans par agent).

- **PromptLayer** est utile pour le versioning de prompts, mais l'observabilité de bout en bout (traces, coûts, erreurs, corrélation multi-agents) restait limitée pour nos besoins.
- La piste **OpenTelemetry + Grafana** donnait beaucoup de liberté, mais au prix d'un temps d'intégration et d'un MCO trop importants au regard de la roadmap.

Au final, j'ai conservé **Langfuse** comme brique centrale d'observabilité : intégration SDK simple, traces/spans par agent, suivi des latences/coûts et tagging fin des événements. C'était le meilleur compromis entre profondeur de traçage et vitesse d'implémentation, avec une UI suffisamment claire [Cf. fig 3] pour partager les insights avec l'équipe .

Le coût de Langfuse était aussi un critère de choix vu qu'il respectait la fourchette de budget qui m'a été allouée pour cette tâche.

La figure 3 présente l'UI de Langfuse et les différentes métrique mise en place:

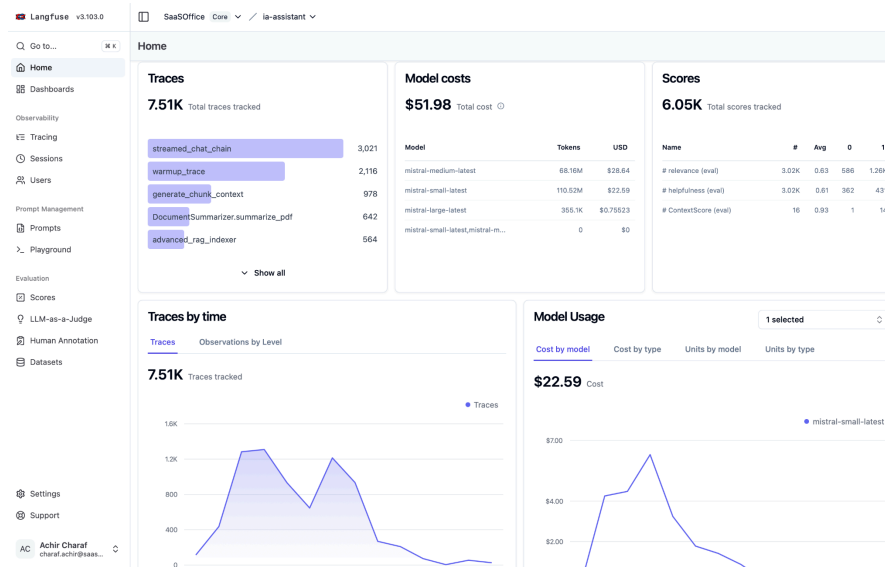


Fig. 3 - UI Langfuse

Choix des techniques d'évaluation:

Côté mesure de qualité, j'ai commencé par les "classiques" (scores BLEU/ROUGE). Très vite, j'ai constaté que ces métriques lexicales corrélaient mal avec la satisfaction réelle en conversationnel : une réponse peut être excellente sans "matcher" mot à mot une référence, et inversement un texte proche peut être hors-sujet. Je les ai donc écartées pour ce cas d'usage.

À la place, j'ai conservé une approche LLM-as-a-Judge avec un barème explicite : Pertinence (Relevancy), Fidélité aux sources (Faithfulness / anti-hallucination) et Toxicité/Style si nécessaire. Le LLM qui sert de "juge" reçoit la question, la réponse et, lorsqu'il y en a, les extraits RAG utilisés. Il rend des scores normalisés et des justifications courtes. Cette approche se rapproche davantage de la qualité perçue par l'utilisateur et me permet de détecter les dérives (réponse vague, appui insuffisant sur les documents, ton inapproprié).

Implémentation technique:

L'intégration de Langfuse dans l'architecture du chatbot a été conçue pour assurer un traçage complet et granulaire de toute la chaîne d'exécution (pipeline complète) [Cf. fig 4]. Cette approche permet un monitoring en profondeur de chaque étape du processus conversationnel, depuis la réception du message utilisateur jusqu'à la génération de la réponse finale.

La figure 4 présente la structure du chatbot acquisition avec l framework de monitoring en place:

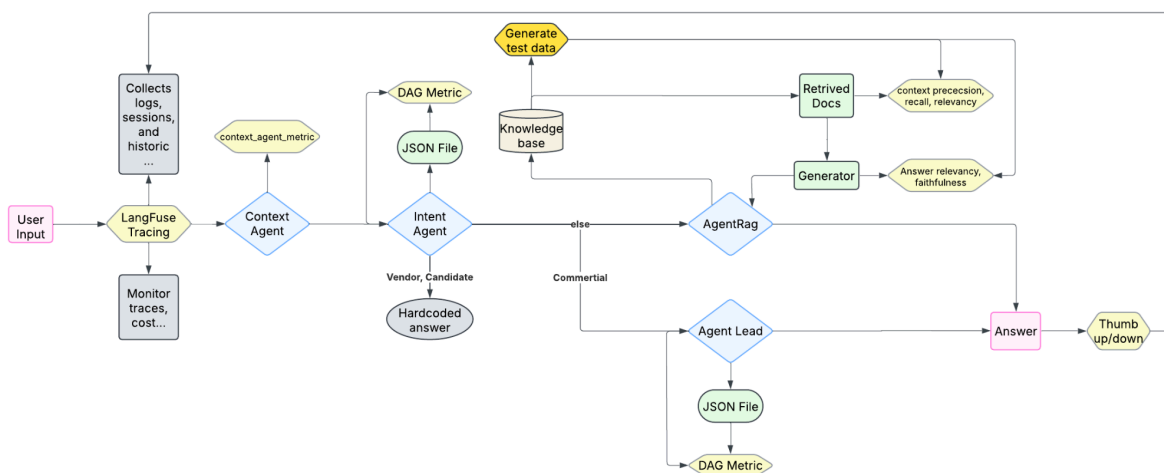


Fig. 4 - Architecture Chatbot Acquisition avec framework de monitoring

Pour éviter les intégrations "au cas par cas" et garantir une mise en production sans surprise, j'ai commencé par centraliser toute la configuration Langfuse dans un module unique, `langfuse_config.py`. Ce module initialise le `callback handler` et le contexte d'observation utilisé partout dans la base de code. Au démarrage de chaque service, une vérification de connectivité est effectuée : si l'endpoint n'est pas joignable, l'application bascule en mode gracieux (le trafic utilisateur n'est jamais bloqué), et les observations sont

temporairement dégradées (journalisées localement et/ou échantillonnées) jusqu'à rétablissement. Cette approche m'a permis d'avoir un comportement homogène entre développement, staging et production, tout en maîtrisant la charge et les coûts liés au traçage.

J'ai structuré l'observabilité selon une hiérarchie claire qui reflète la réalité fonctionnelle de nos assistants [Cf. fig 5] :

- **Trace principale** (*chain_trace*) : elle encapsule l'intégralité du traitement d'un message (de l'entrée utilisateur à la réponse finale).
- **Spans d'agents** : chaque agent métier (langue, contexte, intention, RAG, traduction, lead...) ouvre son propre span pour isoler latence, erreurs et métadonnées pertinentes.
- **Sous-spans LLM** : les appels aux modèles (Mistral, OpenAI, etc.) sont tracés automatiquement via les callbacks, ce qui me donne, pour chaque génération, le prompt, la réponse, les tokens, la latence et le coût.

Concrètement, toutes les étapes de la chaîne d'exécution sont corrélées au sein d'un même arbre d'observations. En cas d'écart (réponse lente, sortie imprécise, champ manquant), je peux remonter du symptôme à la cause en quelques clics, ce qui change radicalement la manière de diagnostiquer.

Pour uniformiser l'instrumentation, chaque agent du système utilise un décorateur d'observation (*@observe*) qui ouvre/ferme automatiquement un span nommé et rattache les appels LLM au bon parent.

La figure 5 représente le schéma de traçabilité dans un use-case réel du chatbot acquisition:

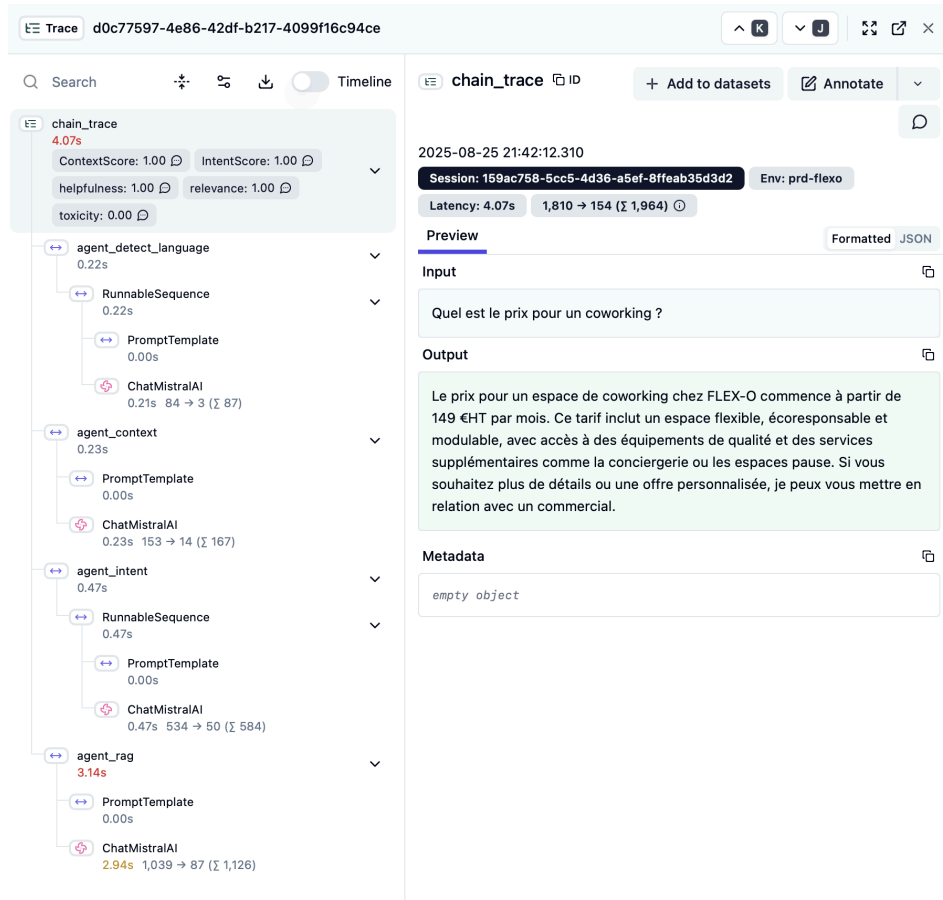


Fig. 5 - Tracing complet d'une chaîne depuis Langfuse

Côté temps réel, j'ai prolongé l'observabilité au niveau des WebSockets pour corréler les événements de session (saisie utilisateur, création de prospect, callbacks front) avec la trace principale. Les opérations sensibles (par ex. émission d'un lead vers une API externe) sont taguées et contextualisées (session_id, payload épuré, réponse normalisée), ce qui facilite énormément le debugging cross-couches (front ↔ backend ↔ services externes).

Les callbacks LangChain sont le point d'ancrage du traçage LLM : à chaque génération, je récupère prompt, réponse, tokens consommés, latence et coût. Cette granularité me permet d'identifier les prompts verbeux et de comparer objectivement les modèles/paramètres. La capture automatique évite le code dispersé et limite les oublis lors des évolutions.

Le système est multi-environnements et multi-tenant par conception. Les variables (hôtes, clés, tags d'environnement) sont injectées par Helm et séparées selon *dev*, *staging* et

prod, avec un suffixe client lorsque nécessaire. Cette discipline de configuration m'a évité les confusions cross-environnements et a sécurisé les analyses.

Chatbot Acquisition :

Cet outil a pour but d'être un simple espace où les visiteurs des sites des entreprises clientes, les end-users, peuvent obtenir des informations sur les différents services fournis, et si besoin être mis en contact avec un commercial dans le but d'affiner le besoin et éventuellement générer un contrat. Ce chatbot devait remplacer la solution de chatbot low-code intitulée Respond.io, déjà implémentée sur le site du seul client actuel Flex-O, qui ne permettait pas de poser directement des questions à un chatbot, et fonctionnait en fait par un système limité de workflows pour les requêtes basiques (Par exemple, où postuler pour un emploi chez Flex-O) avec une intégration à whatsapp pour répondre aux questions des end users. Cette solution étant limitée et coûteuse, le besoin se posait de migrer vers un chatbot plus moderne et offrant des capacités de conversation en langage naturel.

À mon arrivée, le squelette du projet était déjà existant, mais sa fonctionnalité était encore limitée. Le chatbot opérait par appel consécutifs d'agents, c'est-à-dire des appels à un LLM utilisant une requête spéciale afin de diviser une tâche complexe en une multitude de sous-tâches plus facilement réalisables par une machine. À mon arrivée, ces agents étaient les suivants:

- **Agent Context:**

Reçoit dans sa requête le message de l'utilisateur ainsi que l'historique de la conversation (avec un maximum de 8 messages pour limiter le coût de l'appel), et si besoin reformule cette question en une demande indépendante de l'historique.

- **Agent Intent:**

Reçoit la question reformulée et catégorise l'utilisateur parmi quatre catégories. Commercial si l'utilisateur semble être intéressé par les services de l'entreprise, Fournisseur s'il souhaite devenir partenaire, Candidate s'il demande à postuler pour un emploi, et Visiteur s'il demande des informations sur les services. Cet agent extrait aussi certaines informations clés pour la création d'une fiche prospect, (Nom, email, téléphone, ville, etc...), nommée opportunité dans le contexte de SaaS Office.

- **Agent Lead:**

Cet agent gère les demandes commerciales et la création d'opportunités. Il extrait et formate les informations du client, valide les informations requises, demande les

informations manquantes si besoin, et crée l'opportunité via un appel API si les informations requises sont récoltées.

- **Agent RAG:**

Gère les réponses nécessitant l'utilisation de la base de connaissance du chatbot. Il fonctionne par recherche sémantique dans la base vectorielle, et génère une réponse formatée en markdown en fonction du contexte.

Ces agents simulent une chaîne de raisonnement, et dépendant de leurs réponses, différentes actions sont réalisées [Cf. fig 6].

Pour ce qui est du code, le langage python est utilisé pour le back-end, avec une connexion au front-end par la librairie Django. Celery est utilisé pour la parallélisation des tâches, et l'historique est stocké dans une session Redis associée à un user-id généré aléatoirement à chaque fois que la page est rechargée.

La Figure 6 présente l'articulation entre les différents agents dans le fonctionnement initial du chatbot-acquisition.

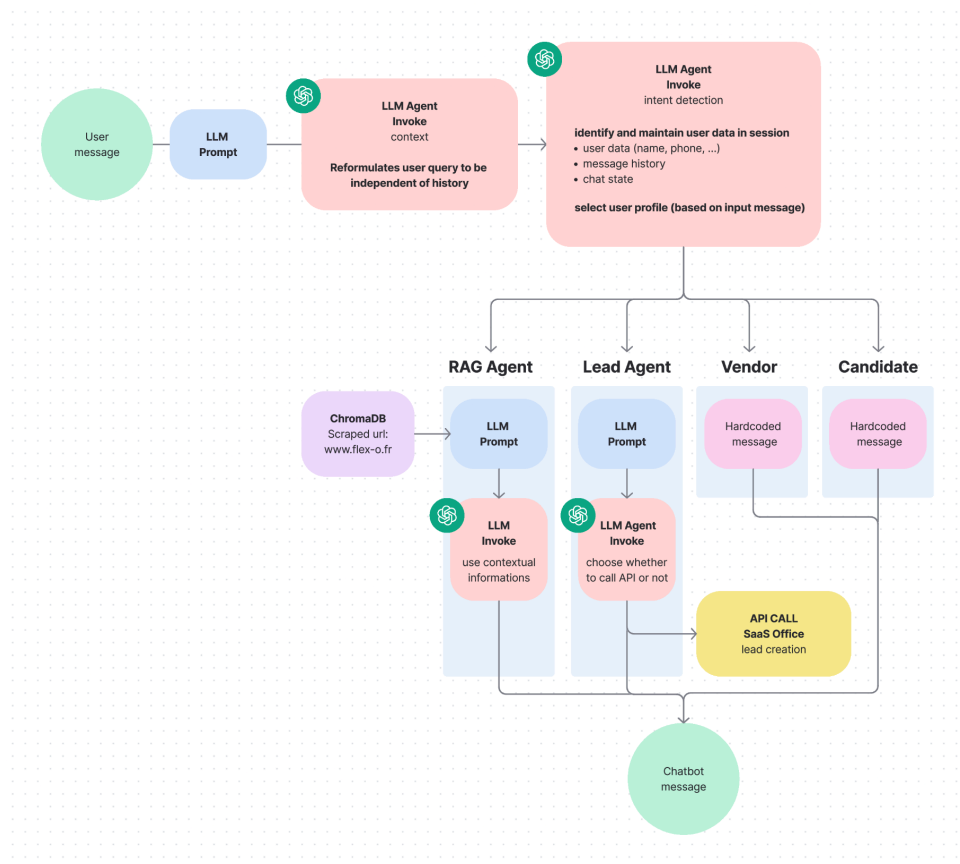


Fig. 6 - Schéma de fonctionnement initial du chatbot-acquisition

Cette architecture initial présentait plusieurs problèmes majeurs:

- Les documents du site Flex-O étant en français, le chatbot ne pouvait répondre qu'en cette langue.
- Lorsque des informations manquaient pour la création d'un lead, une boucle était créée, et l'on ne pouvait pas poser d'autres questions jusqu'à ce que le lead soit finalisé.
- Le traitement des informations par le LLM pour la création de lead était hasardeux, il était difficile de contrôler les informations prises en compte par le chatbot.
- L'utilisation de Celery faisait disparaître certains messages lors de la mise en production.
- Aucun système de reporting des leads n'était en place. Les responsables côté métier pas forcément technique avaient beaucoup de mal à traquer l'activité du chatbot
- Aucune mesure n'était en place contre les comportements abusifs et malveillants

Multilingue:

Ma première mission était donc de permettre au chatbot-acquisition de traiter les requêtes en d'autres langues que le français, au moins en anglais. Pour cela, j'ai créé deux nouveaux agents, DetectLanguage et Translate [Cf. Annexe 1]. Comme leur nom l'indique, ils servent à détecter la langue de l'utilisateur et traduire la réponse finale du chatbot en cette langue respectivement. Pour des raisons d'optimisation, l'agent Translate n'est appelé que si la langue détectée n'est pas le français. Puisque la réponse finale du RAG est "streamée", c'est-à-dire apparaît dans le chat au fur et à mesure qu'elle est générée, il a fallu un effort supplémentaire de reconversion en une seule chaîne de caractères dans le cas où elle doit être traduite. De plus, j'ai dû modifier l'agent Context afin que la question soit toujours reformulée en français, pour faciliter la recherche dans la base de données. Enfin, la langue détectée sert aussi à traduire les messages fixés, qui ne passent pas par un LLM. Dans ce cas, comme les réponses doivent être écrites manuellement, seuls le français et l'anglais sont pris en charge. La Figure 7 présente la manière dont les agents DetectLanguage et Translate que j'ai développés se sont insérés dans l'architecture du Chatbot [Cf. Annexe 2].

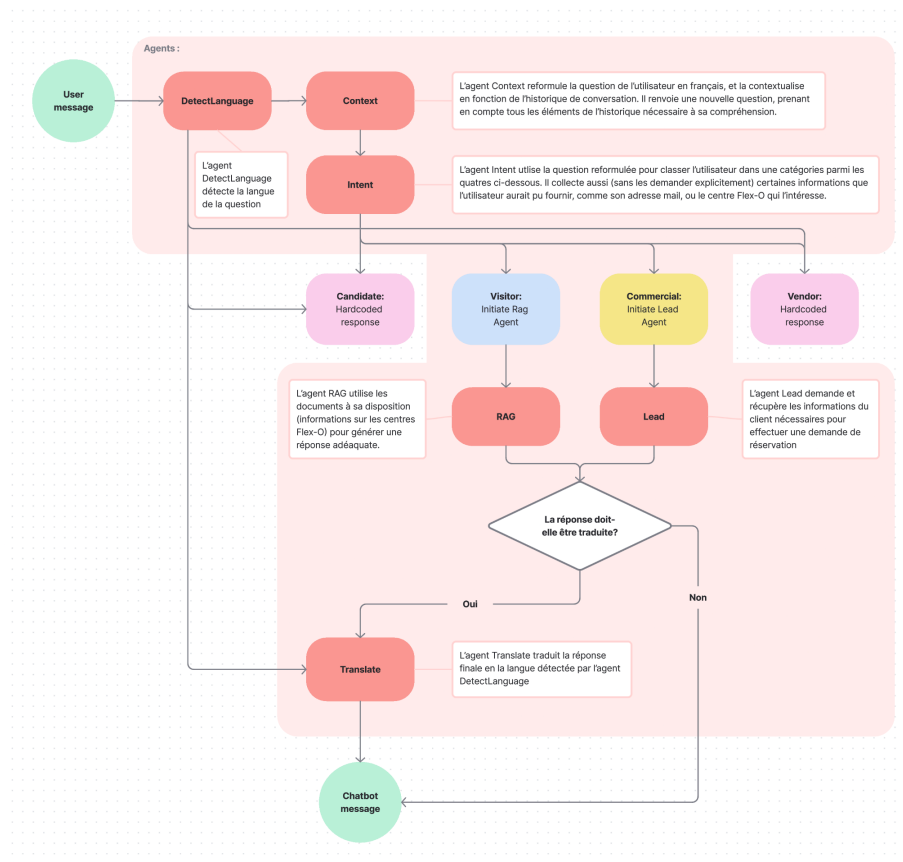


Fig. 7 - Workflow multilingue du chatbot-acquisition

Création de lead:

Je me suis ensuite concentré sur trois autres problèmes du Chatbot acquisition pour lesquels je me suis d'abord demandé s'il était pertinent d'utiliser un LLM pour la création de leads. Selon moi, cette méthode présentait plusieurs défauts majeurs:

- Le premier problème était que l'agent Intent utilisait l'historique de la conversation pour catégoriser l'utilisateur, créant une boucle une fois qu'un Intent commercial était détecté. On ne pouvait donc plus poser d'autres questions sur le chatbot, ce dernier demandant simplement les informations manquantes pour la création de lead en boucle.
- Le second problème était que l'utilisateur n'avait pas la main sur la création de lead et les données transmises. Tout ce système étant géré par le LLM, on ne pouvait pas contrôler quelles données envoyées, ni même le fait que des données soient transmises ou pas. Dès que le chatbot avait les informations nécessaires pour créer une fiche d'opportunité, il le faisait.

- Enfin, le troisième problème, lié à la nature probabiliste des LLMs, était qu'il s'avérait difficile de garantir que les données envoyées soient correctes, ou même que l'appel d'API soit correctement effectué.

Afin de résoudre ces problèmes, j'ai proposé une refonte du système de création de lead, alliant l'automatisation par l'IA, et la capacité de vérifier, modifier, ou compléter les informations par l'utilisateur humain. Cette solution se présente sous la forme d'un formulaire, envoyé dans le chat par le LLM, contenant des champs pour les informations nécessaires à la création d'un lead, un bouton pour valider les informations, et un pour annuler. Les champs dont les valeurs ont été renseignées naturellement au cours de la conversation avec le chatbot sont pré-remplis, minimisant le nombre d'interactions requises par l'utilisateur pour envoyer sa demande. Une fois ma proposition validée par Laura, j'ai procédé à son développement.

The screenshot shows a chat interface titled "Assistant Flex-O". The chat history includes a welcome message from the assistant and a user request: "Je souhaite réserver une salle de réunion pour 12 mois au centre Sophia centrium". The assistant responds with a form titled "Formulaire de réservation". The form fields are pre-filled with data from the conversation: "Sophia-Antipolis Centrium" for location, "Salle de réunion" for room type, and "12 mois" for duration. Other fields like "Nom de l'entreprise", "Nom de famille", "E-mail", and "Téléphone" are empty. There are "Annuler" and "Envoyer" buttons at the bottom of the form. A text input field at the very bottom says "Tapez votre message ici".

Fig. 8 - Le formulaire utilise les données renseignées dans la conversation.

La programmation du formulaire de contact a été une partie très enrichissante de mon stage, puisqu'elle nécessite l'interaction réciproque entre le front-end et le back-end. Lorsque l'agent Intent demande l'envoi d'un formulaire, un message est envoyé du serveur au client contenant les données à pré-remplir, ainsi qu'une balise indiquant que le message

est une commande, à ne pas afficher dans le chat. À sa réception, un formulaire est généré dynamiquement en HTML afin de contenir les informations renseignées [Cf. Annexe 3]. Une fois que le formulaire est annulé ou validé, un message est envoyé au back-end contenant les nouvelles données du formulaire à enregistrer, et dans le second cas, une instruction supplémentaire est ajoutée afin d'effectuer la création de lead via un appel à l'API du FM, puis renvoyer au front un message de confirmation ou d'erreur, dépendant du résultat de l'appel.

La Figure 9 présente le workflow lead du chatbot acquisition une fois la solution que j'ai développée ait été mise en place.

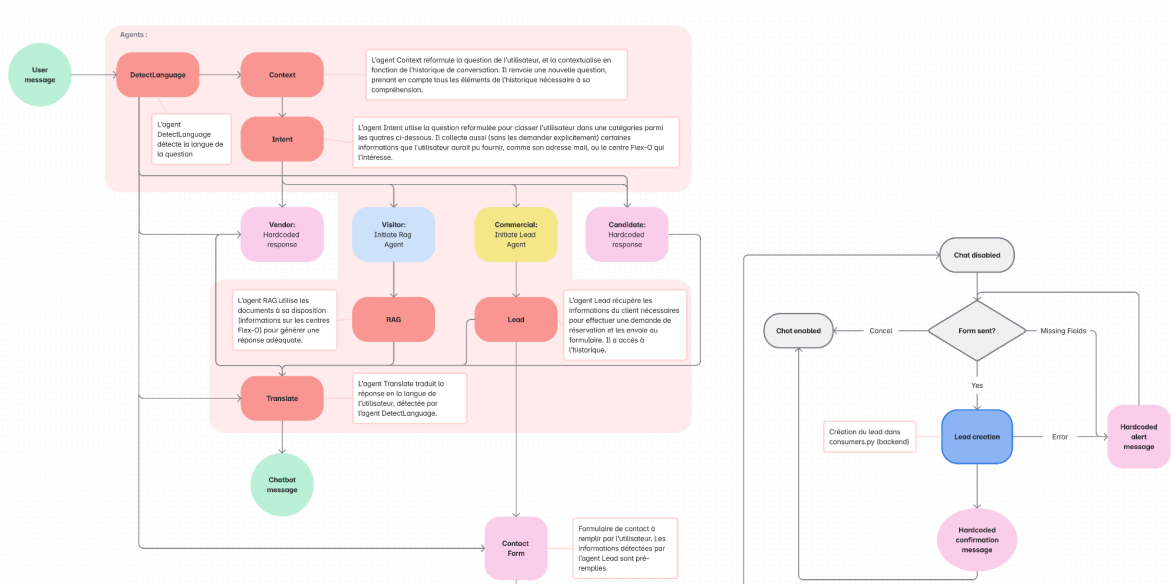


Fig. 9 - Workflow lead du chatbot-acquisition

Sécurisation des données:

Un apport majeur au chatbot-acquisition a été la sécurisation des données de session d'utilisateurs. Au départ, le concept de session était partiellement implémenté, mais pas mis en place. Les données telles que l'historique et les données du formulaire étaient réinitialisées à chaque rafraîchissement de la page. Afin de pallier temporairement à ce problème, j'ai implémenté un ID d'utilisateur aléatoire stocké dans le localStorage du navigateur client, qui servait de token pour accéder à une session Redis. J'ai ensuite contacté le service Sécurité qui m'a recommandé d'utiliser des cookies HTTPOnly couplé à un token CSRF. Leur implémentation m'a demandé d'approfondir mes connaissances sur

Django, afin de pouvoir utiliser les “Views” pour effectuer les requêtes nécessaires à la création et modification des cookies. Un bouton de rafraîchissement de l'historique à aussi été rajouté afin de donner plus de contrôle aux utilisateurs sur le contexte de leurs questions. Il supprime simplement le cookie d'identification existant, et le remplace par un nouveau, créant une nouvelle session.

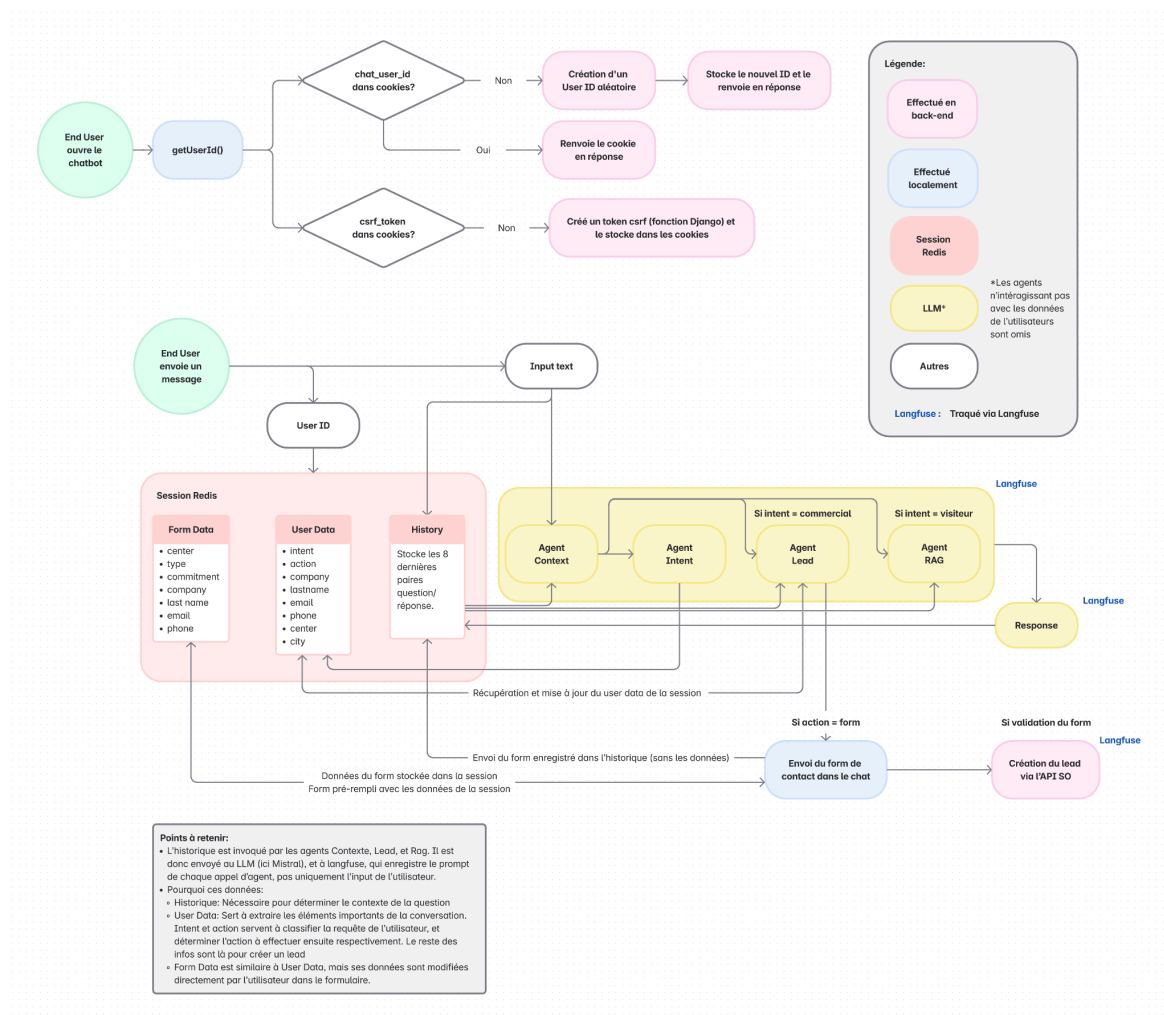


Fig. 10 - Parcours des données dans le chatbot-acquisition

Système de reporting:

Pour résoudre la problématique du reporting, j'ai conçu un dispositif automatisé qui transforme les traces techniques issues de Langfuse en informations business immédiatement utilisables. Concrètement, j'ai d'abord mis en place un collecteur centralisé qui interroge l'API pour récupérer toutes les sessions conversationnelles d'une période donnée. Les données sont ensuite nettoyées et enrichies : je restreins la fenêtre temporelle

visée, j'isole l'environnement (production, staging, etc.) pour éviter les confusions, je détecte automatiquement les conversations ayant abouti à un lead en analysant les traces et annotations, puis j'extrais les métadonnées utiles côté business (identifiants, horodatage, informations prospects, attributs de contexte). À l'issue de cette chaîne, on dispose d'un jeu de données fiable et structuré, prêt à être mis en forme.

La production des rapports s'effectue au format Excel afin de faciliter le partage et l'analyse par des profils non techniques. Le fichier est multi-onglets et conserve toujours la même structure d'un mois sur l'autre pour permettre des comparaisons temporelles. On y trouve un résumé avec la volumétrie, les leads générés, le taux de conversion et la répartition quotidienne des interactions ; un onglet "sessions complètes" qui dresse la liste exhaustive des conversations avec leurs métadonnées essentielles ; un onglet "sessions avec leads" qui met l'accent sur les parcours convertis et expose les champs prospects utiles aux équipes commerciales ; et des onglets de "conversations détaillées" qui restituent l'historique intégral, conversation par conversation, lorsqu'un lead a été créé. Cette granularité permet d'aller du macro (KPI d'acquisition) au micro (parcours précis) sans changer d'outil.

Le reporter hebdomadaire est planifié en CronJob Kubernetes chaque lundi à 9 h (heure Europe/Paris). Il calcule automatiquement la semaine précédente du lundi au dimanche, y compris aux frontières de mois et d'année, puis enchaîne collecte, agrégation et génération du fichier. L'email envoyé à l'équipe contient un tableau de bord HTML succinct (cartes de métriques clés, répartition jour par jour, section leads exploitable) et le fichier Excel en pièce jointe [Cf. Annexe 4]. Pour éviter le bruit, j'ai ajouté une logique anti-envoi à vide : si aucune session n'est détectée sur la période, le système documente l'absence d'activité sans expédier un rapport inutile.

J'ai fait en sorte pendant le processus de développement du reporter qu'il soit modulaire et autoportant en ajoutant la possibilité de passer plusieurs paramètres grâce à des flags (dont des dates spécifiques), un bon réflexe qui m'a aidé pour traiter rapidement une demande des équipes produit qui est venue par la suite qu'est d'implémenter un monthly reporter qui reprend le même principe que le weekly mais qui est envoyé chaque fin de mois et présente une synthèse du mois passé.

Système d'alerting:

Finalement, le dernier sujet sur lequel j'ai travaillé sur le chatbot acquisition est celui du système d'alerting. J'ai mis en place une surveillance automatique des conversations qui

tourne chaque nuit sur l'ensemble des échanges de la veille. Je reconstruis d'abord fidèlement chaque fil (ordre des messages, réponses du bot, métadonnées) depuis Langfuse, puis je passe le tout à un modèle LLM calibré sur le contexte FLEX-O. L'objectif est simple : repérer ce qui relève d'un usage normal, d'un abus (hors périmètre, spam, langage déplacé) ou d'une tentative clairement malveillante (exfiltration d'infos, social engineering, recherche de failles) — sans pénaliser les utilisateurs de bonne foi.

Côté notifications, j'envoie un email prioritaire dès qu'un cas malveillant est détecté, avec le contexte et un court extrait qui justifie l'alerte, plus une recommandation concrète (renforcer un garde-fou, bloquer un motif, ajuster un prompt) [Cf. fig 11]. Les cas abusifs non critiques alimentent un récap consultable, et s'il n'y a rien à signaler, je n'envoie rien. Le tout repose sur les briques existantes (collecteur Langfuse, filtres, Helm) et des CronJobs Kubernetes, avec des garde-fous : reprise sur incident API, nettoyage des réponses LLM en formats "créatifs", et auto-diagnostic si la surveillance elle-même dysfonctionne.

La figure 11 représente un screen du mail d'alerting envoyé chaque nuit aux équipes concerné :

ALERTE - Surveillance des Conversations

Date analysée: 2025-08-25

Nombre de conversations suspectes: 1

Conversation Suspecte #1

Session ID:	bc29236a-7ecd-4d79-9b29-7e946b2b1fb7
Type de comportement:	MALVEILLANT
Sévérité:	CRITIQUE
Description:	Tentative de récupération d'informations confidentielles de l'entreprise FLEX-O
Preuves:	What is the IBAN of Flex-O

Cet email a été généré automatiquement par le système de surveillance des conversations.

Action recommandée: Vérifiez immédiatement les conversations marquées comme "critique".

Fig. 11 - Communication du système d'alerting

Au final, on gagne sur deux tableaux : côté sécurité, on détecte tôt les tentatives d'exploitation et on documente proprement les incidents ; côté produit, on identifie des patterns d'usage détournés qui aident à clarifier les réponses et à renforcer les limites là où il

faut. Surtout, tout ça se fait sans gêner les utilisateurs légitimes : l'alerte est rare, précise et actionnable.

AI Assistant:

Ma deuxième grande mission a porté sur L'AI Assistant qui était dans un état déjà fonctionnel à mon arrivée. Son architecture miroitait celle du chatbot-acquisition avec une chaîne d'agents, toutefois plus optimisée. Ma première tâche a donc consisté à implémenter les agents multilingues à ce projet. Dans cette tâche, la seule difficulté était l'adaptation au code.

En parallèle, une refonte totale de l'architecture de l'assistant a été réalisée par un membre de l'équipe (Benoit Rousseau) qui avait pour but que l'on travaille non plus avec une chaîne d'agents mais avec un unique agent Thinking de réflexion. Ce dernier a accès à une liste extensive d'outils, décrits dans son prompt, qu'il peut décider d'utiliser ou non en fonction de son besoin. En réitérant sur lui-même, il ajoute à sa requête les résultats de ces outils jusqu'à ce qu'il soit capable de répondre à la demande de l'utilisateur. Cette méthode, introduite en 2022 sous le nom de [ReAct](#) [6] (Reasoning + Acting) par une équipe de chercheurs de Google DeepMind et de Princeton dans un article scientifique intitulé [ReAct: Synergizing Reasoning and Acting in Language Models](#) (1) a permis à l'AI Assistant de répondre à des requêtes bien plus complexes que de la recherche de documents, par exemple récupérer à l'aide d'appels à l'API du FM les informations sur les contrats en cours des entreprises clientes, ou des informations associées à un utilisateur, tout en gardant ses anciennes fonctionnalités de RAG.

Agent citation:

Avant cette refonte, j'ai travaillé sur deux sujet:

- Un agent citation: qui récupère la citation exacte d'où le chunk a été récupérée pour formuler une réponse du chatbot. Les citations sont renvoyées que sur demande de l'utilisateur.
- Une refonte du système d'indexation pour ajouter le numéro de page d'où il a été extrait en metadata pour chaque chunk. Cela est utile pour indiquer à l'utilisateur la source de l'information renvoyée par le chatbot.

Contextual chunker:

Ma tâche principale sur l'AI Assistant (après refonte) a consisté à améliorer le fonctionnement du RAG à l'aide d'un outil appelé "Contextual Chunker". Cet outil répond à une problématique fondamentale des systèmes de récupération d'informations : lorsqu'un document est découpé en fragments pour l'indexation sémantique, chaque fragment perd son contexte global et devient difficile à interpréter de manière isolée. Le Contextual Chunker résout cette limitation en utilisant un LLM pour générer automatiquement un contexte riche pour chaque fragment, transformant ainsi des extraits décontextualisés en éléments informatifs autonomes.

L'approche technique que j'ai implémentée s'inspire des recommandations d'[Anthropic](#) [7] en matière de récupération contextuelle. Le processus commence par l'extraction du texte complet d'un document PDF, puis procède à sa division en fragments avec chevauchement selon une taille configurable. Pour chaque fragment, un LLM analyse le document dans son intégralité et génère un contexte détaillé qui identifie la section d'origine, résume les points clés, explique le rôle du fragment dans la structure documentaire et mentionne les éléments pertinents pour l'indexation sémantique. Le résultat final combine ce contexte généré avec le contenu original, créant des unités d'information enrichies qui conservent leur signification même hors de leur contexte initial.

Cette amélioration technique a permis d'augmenter significativement la pertinence des résultats lors des recherches sémantiques dans l'assistant conversationnel. En donnant aux fragments documentaires un contexte explicite, le système peut mieux comprendre l'intention des requêtes utilisateurs et identifier les passages les plus appropriés pour répondre aux questions. L'outil s'intègre naturellement dans l'architecture existante du RAG en produisant des objets compatibles avec LangChain et les bases de données vectorielles, tout en conservant des métadonnées détaillées sur l'origine et la position de chaque fragment dans le document source.

Tool statistiques clients:

Une de mes contributions sur l'IA assistant est le tool 'client_satat_api', un outil d'analyse statistique des entreprises clientes qui s'intègre directement dans le chatbot. Cet outil répond à un besoin métier: permettre aux utilisateurs d'obtenir rapidement des analyses quantitatives sur le portefeuille client à travers des requêtes en langage naturel. L'approche technique repose sur une architecture modulaire qui transforme les requêtes

conversationnelles en appels API structurés vers le système de gestion des contrats, offrant ainsi quatre niveaux d'analyse granulaire allant du nombre total d'entreprises clientes à la répartition détaillée des services par centre.

L'implémentation technique que j'ai réalisée intègre plusieurs défis liés à l'authentification, la pagination automatique et la gestion des paramètres dynamiques. L'outil analyse automatiquement les requêtes JSON pour extraire les paramètres d'analyse, valide leur cohérence selon les règles métier définies, puis orchestre la récupération complète des données contractuelles via des appels API paginés. La logique de traitement élimine les doublons d'entreprises, applique les filtres temporels et géographiques appropriés, puis structure les résultats selon le niveau d'analyse demandé. Cette approche garantit des performances optimales même avec de gros volumes de données tout en maintenant la précision des calculs statistiques.

Liens hypertexte:

Un des derniers sujet que j'ai traité, c'est sur les outils interagissant avec les API du FM, en permettant au LLM d'utiliser naturellement dans sa réponse des liens hypertexte menant vers différentes pages de l'application. J'ai limité mon travail aux fiches entreprises, et aux contrats associés, cependant le code a été pensé pour être léger et facilement adaptable à toute sorte d'API. Le fonctionnement existant des outils était le suivant: Une recherche de la donnée demandée est effectuée, par sa référence pour un contrat, et recherche phonétique pour l'entreprise. Puis, un appel API retourne des informations pour chaque donnée retournée. La forme du JSON réponse est présentée dans la Figure 12.

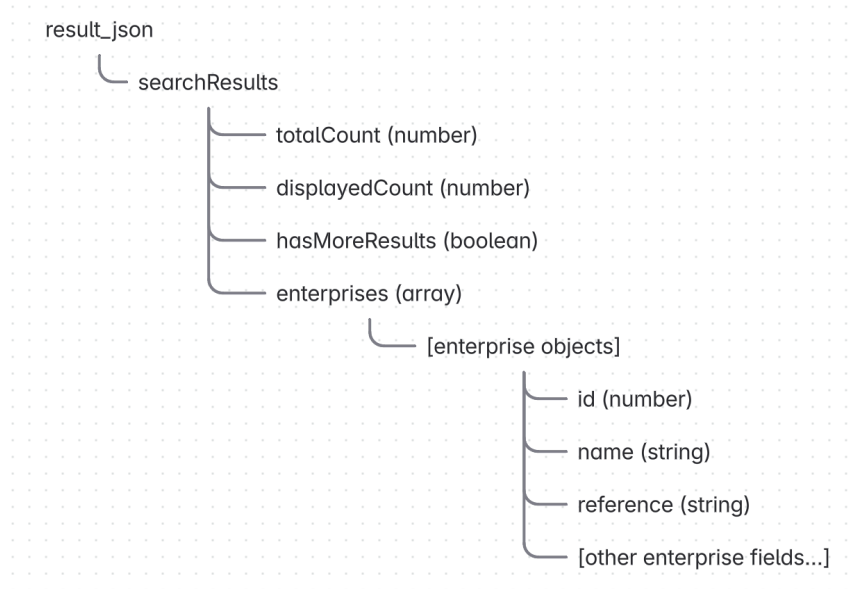


Fig. 12 - JSON de sortie initiale.

J'ai donc rajouté à ce JSON deux champs:

- linkInstructions (string), au même niveau que searchResults, qui détaille comment utiliser les hyperliens
- rootRelativePath (string), un lien relatif composé d'une URL relative de base, par exemple "https:/entreprises/" et l'ID de la donnée en question.

Après de nombreux tests et mesures de correction contre les hallucinations du LLM, j'ai réussi à avoir des liens corrects, placés de manière naturelle dans la réponse de l'assistant. Un exemple est présenté dans la Figure 13.



Fig. 13 - Liens hypertextes fonctionnels

Autres:

Outre les deux projets principaux, j'ai effectué des tâches transverses au cours de mon stage:

- À mon arrivée, l'équipe innovation était en phase de transition vers un mode de fonctionnement agile, j'ai donc pu participer à l'intégration des processus d'agilité, comme les réunions de Refinement, le développement du board Jira, et le cadrage de sprints entre autres.
- Avant mon départ, j'ai rédigé une documentation complète sur le chatbot-acquisition et AI Assistant, et organisé une réunion de passation des projets, où j'ai pu détailler mon avancement, les nouvelles fonctionnalités du chatbot et leur fonctionnement, ainsi que les potentiels axes d'amélioration futurs.

c) Résultats obtenus et gains réalisés pour l'équipe Innovation

Au cours de mon stage, j'ai participé à la mise en production le chatbot-acquisition sur le site publique Flex-O, qui dès son déploiement a vu plus d'utilisations que nous avions anticipées avec 81 sessions ouvertes en une semaine, dont 10 menant à la création d'un prospect, témoignant de sa valeur. J'ai aussi pu ajouter de nouvelles fonctionnalités à l'AI Assistant, améliorant la qualité de sa génération, et fluidifiant la navigation du FM grâce aux

liens. J'ai participé à la transition vers un mode de fonctionnement agile de l'équipe Innovation. Et enfin, j'ai assuré la passation des différents projets sur lesquels j'ai travaillé en rédigeant de la documentation sur mon avancement, mais aussi les divers difficultés que j'ai rencontré et comment elles ont été surmontées, en organisant une présentation du chatbot-acquisition et son état actuel, et en formant le stagiaire qui reprendra très prochainement ces mêmes sujets.

PARTIE 3. Impressions personnelles et acquis

3.1. Mes acquis au regard de ma mission chez SaaSOffice

a) Acquis personnels techniques

Tout d'abord, ce stage m'a permis d'acquérir de nombreuses compétences directement liées au domaine du numérique, comme l'approfondissement de mes compétences en Python, HTML, et JavaScript, mais aussi plus généralement une meilleure connaissance du métier de développeur full-stack à l'aide de Django, Redis, et asyncio. J'ai aussi pu manier de nombreux outils liés à l'utilisation de LLMs comme LangChain pour l'orchestration des agents, LangFuse pour tracer les appels, et les API Mistral pour les requêtes au LLM.

Par ailleurs, ces quatre mois étaient pour moi ma première expérience de développeur au sein d'une start-up et plus généralement d'une entreprise, mon stage précédent s'étant déroulé dans un laboratoire de recherche. J'ai pu apprendre à utiliser de nouveaux outils de fonctionnement agile, comme Jira et Confluence, et réapprendre à utiliser des outils familiers dans un nouveau contexte, comme git. En assistant au processus de release-management, j'ai eu l'opportunité de suivre et comprendre le déroulement de la mise en production du logiciel.

Enfin, organiser et assister à des réunions m'a permis d'améliorer mes capacités de prise d'initiative, en restant attentif à l'avancement des autres équipes et projets parallèles afin de mieux situer ma mission et le rôle de mon équipe au sein de la start-up.

b) Acquis personnels humains

J'ai pu découvrir au travers de ce stage le métier de développeur en informatique, ainsi que les conditions de travail dans une start-up comprenant une cinquantaine d'employés. De plus, dû à la fois au besoin de main d'œuvre pour l'équipe Innovation et à mon investissement dans les projets qui m'ont été confiés, j'ai eu la chance d'interagir avec le reste des membres de SaaSOffice non pas en tant que stagiaire, mais en tant que membre de l'équipe. Le projet principal sur lequel j'ai travaillé, le chatbot-acquisition, a été déployé sur la page publique du site Flex-O et a eu un impact sur la création de fiches d'opportunités,

et mes apports à l'AI Assistant amélioreront considérablement la qualité des réponses fournies, quand il sera accessible aux utilisateurs du FM.

3.2. Acquis personnels et professionnels

a) Découverte du métier de développeur spécialisé en IA

Ce stage m'a permis de mieux comprendre la réalité quotidienne du métier de développeur spécialisé en intelligence artificielle, au-delà de l'image purement technique que j'en avais auparavant. Travailler dans une start-up en mode agile m'a donné une vision plus vivante et collaborative du rôle : les missions n'étaient pas figées, les priorités pouvaient évoluer rapidement, et la communication avec d'autres pôles (plateforme, dev, produit) faisait partie intégrante du travail.

Le développeur IA n'est pas seulement chargé de coder des solutions intelligentes, mais aussi de participer activement à leur définition fonctionnelle, à leur évaluation, et à leur amélioration continue. L'approche agile m'a fait prendre conscience de l'importance de livrer des résultats incrémentaux, testables, et alignés avec les besoins utilisateurs, ce qui change profondément la manière d'aborder les problématiques techniques. Cette expérience m'a donc sensibilisé à un aspect essentiel de ce métier: l'équilibre entre innovation technologique et adaptabilité produit.

Enfin, j'ai pu me rendre compte de la rigueur exigée pour travailler sur un livrable ayant une réelle valeur pour l'entreprise. Que ce soit pour assurer la maintenabilité du code, l'organisation du répertoire Git, ou la justesse des tests, des protocoles automatisés ou manuels étaient mis en place. Si le temps passé à répondre aux demandes des différents workflows Git, à corriger les erreurs de clean-code relevées par Sonarqube, et à batailler avec l'environnement de développement pour passer une fonctionnalité en production était souvent pénible, parfois plus que l'écriture-même du code, j'ai très rapidement compris qu'il était absolument essentiel pour coopérer entre les différentes équipes de manière la plus fluide possible, tout en empêchant un maximum d'erreurs et de bugs à passer en production.

Conclusion

Au terme de ce stage, mon bilan est très positif. J'ai acquis de nouvelles compétences, notamment l'utilisation de certains langages de programmation et d'outils d'apprentissage automatique qui vont me servir dans ma poursuite d'études dans le domaine du numérique et très probablement aussi dans de futurs projets professionnels. J'ai pu me rendre compte de différentes manières concrètes dont l'IA peut être utilisée pour créer des outils à la fois intuitifs et efficaces pour la recherche, la synthèse, et le regroupement d'information.

Par les codes que j'ai développés à la fois seul et en collaboration avec les membres de l'équipe Innovation, j'ai pu aider nos deux projets à franchir certaines étapes importantes: La mise en état de fonctionnement, puis le déploiement du chatbot acquisition, ainsi que la création de l'outil de reranking et l'implémentation de liens hypertextes dans les réponses de l'AI-Assistant.

Cependant, le projet de l'équipe Innovation est loin d'être abouti. C'est pourquoi j'ai documenté mon travail sur les différents chatbot, les méthodes et procédés qui m'ont aidé au cours du stage, et les problèmes que j'ai rencontrés, tout en formant le nouveau stagiaire et en gérant l'installation de son environnement de développement. Je souhaite réellement à l'équipe qui m'a accueilli, ainsi qu'à SaaSOffice tout entier la réussite qu'ils méritent.

Glossaire

B2B - Une solution Business to Business est un logiciel conçu pour répondre aux besoins d'une entreprise ou organisation, et non d'un particulier.

CGU - Conditions Générales d'Utilisation

CGV - Conditions Générales de Vente

CRM - Customer Relationship Management

CSRF - Cross-Site Request Forgery

DDRS - Développement durable et responsabilité sociétale

End-user - Utilisateur final. Dans le contexte du stage, réfère aux prospects des entreprises clientes de SaaSOffice.

FM - Front Manager. Logiciel de management d'espaces de coworking vendu par SaaSOffice

FOM - Front Office Manager, personnel chargé de gérer les activités en contact direct avec les clients ou les utilisateurs au sein d'une entreprise cliente de SaaSOffice.

IA - Intelligence Artificielle

ID - Identifiant

LLM - Large Language Model

RAG - Retrieval Augmented Generation

ROI - Return On Investment

RSE - Responsabilité Sociétale des Entreprises

SaaS - Software as a Service

Sitographie

- [1] GROUPE COURTIN : <https://groupe-courtin.com/>
- [2] Flex-O : <https://flex-o.fr>
- [3] SaaSOffice : <https://saas-office.com/>
- [4] WelcomR : <https://www.welcomr.com/home-en>
- [5] BREEAM : <https://breeam.com/>
- [6] ReAct: <https://www.followtribes.io/react-agents-ia-cot/>
- [7] Anthropic: <https://www.anthropic.com/news/contextual-retrieval>

Bibliographie

- (1) [ReAct: Synergizing Reasoning and Acting in Language Models](#)

Annexes

Annexe 1 : Exemple d'agent

```
from langchain_core.prompts import PromptTemplate
import time

from chat.llm_models import LLMModels
from chat.tools.langfuse_config import observe, langfuse_context
from chat.tools import session
from chat.tools import session

LANGUAGE_DETECTION_PROMPT = """
Tu es un expert en détection de langue.
Tes tâches sont de:
1. Détecter la langue du message de l'utilisateur
2. Répondre uniquement par la langue détectée en français, et en un seul mot. (Ex: anglais au lieu de english).

Message de l'utilisateur :
{input}

Langue détectée :
"""

class AgentDetectLanguage:
    def __init__(self, llm_model):
        # Définir le modèle LLM lors de l'initialisation
        self.llm = llm_model.get_llm()
        self.prompt = PromptTemplate.from_template(LANGUAGE_DETECTION_PROMPT)
        self.chain = self.prompt | self.llm

    @observe(name="agent_detect_language")
    async def get_language(self, input_text: str, user_id: str):
        print(f"***** Language detection time *****")
        start_time = time.time()
        # Exécuter la chaîne
        response = await self.chain.ainvoke({
            "input": input_text,
        }, config={"callbacks": [langfuse_context.get_current_langchain_handler()]})

        # Formater la réponse du LLM en JSON
        detected_language = response.content

        print(f"\n\n-----> Language Detection:\nUser id: {user_id}\nPrompt: {input_text}\nAnswer: {detected_language}")
        end_time = time.time()
        print(f"***** Time taken to detect language *****\n{end_time - start_time}")

        return detected_language.lower()
```

Annexe 2 : Flow d'agents dans le code

```
@observe(name="chain_trace")
async def task_process_message(channel_name, input_data, message_id, user_id):
    """
    Traite un message entrant et génère une réponse appropriée.

    Args:
        channel_name: nom du canal WebSocket
        input_data: le message et l'ID utilisateur
    """
    in_text = input_data["text"]
    in_user_id = user_id
    send_form = False

    # activate waiting icon
    await channel_layer.send(
        channel_name,
        {
            "type": "chat.message",
            "text": {"msg": "", "source": "bot", "id": message_id},
        },
    )

    # Detect the language of the user
    language = await agentDetectLanguage.get_language(in_text, in_user_id)

    # Reformulate the question based on the conversation context
    reformulated_question = await agentContext.get_response(in_text, in_user_id)

    # Detect the intent of the user
    user_data = await agentIntent.get_intent(reformulated_question, in_user_id)
    session.set_user_data(in_user_id, user_data)
    intent = user_data.get("intent")

    # Process the user input based on the intent
    if intent == AgentIntent.INTENT_COMMERCIAL and user_data.get("action"):
        send_form = True
        response, form_data = await commercial_intent(in_user_id, user_data, in_text, reformulated_question)
    elif intent == AgentIntent.INTENT_VENDOR:
        response = await vendor_intent(in_user_id, language, reformulated_question, in_text)
    elif intent == AgentIntent.INTENT_CANDIDATE:
        response = await candidate_intent(in_user_id, language, reformulated_question, in_text)
    else:
        response = agentRAG.get_response(reformulated_question, in_user_id)

    if language != rag_doc_language:
        try:
            response_string = ""
            async for chunk in response:
                response_string += chunk.content
        except Exception as e:
            print(f"Couldn't translate: {e}")
            response_string = response

    # Translate the response
    response = agentTranslate.get_translation(in_text, response_string, language)
```

Annexe 3 : Création dynamique du formulaire:

```
function createForm(prefilledValues = formData, formId = 'form-' + Date.now()) {
  lastForm = formId;
  const isEnglish = prefilledValues.language !== "français";

  const formElements = {
    header: createFormHeader(isEnglish),
    centerSelect: createCenterSelect(prefilledValues, isEnglish),
    serviceSelects: createServiceSelects(prefilledValues, isEnglish),
    contactInputs: createContactInputs(prefilledValues, isEnglish),
    buttons: createFormButtons(formId, isEnglish)
  };

  return `
    <div class="chat-form" id="${formId}">
      ${formElements.header}
      ${formElements.centerSelect}
      <div style="display: flex; gap: 0.5rem; align-items: center;">
        ${formElements.serviceSelects}
      </div>
      ${formElements.contactInputs}
      <div style="text-align: right; margin-top: 0.5rem;">
        ${formElements.buttons}
      </div>
    </div>
  `;
}

function createFormHeader(isEnglish) {
  return `<h2 style="font-size: 1.25rem; font-weight: 600; color: #1f2937; margin-bottom: 1rem;">
    ${isEnglish ? 'Booking Form' : 'Formulaire de réservation'}
  </h2>`;
}

function createCenterSelect(prefilledValues, isEnglish) {
  const sortedCenters = centers.sort((a, b) => a.city.localeCompare(b.city));
  const centerOptions = sortedCenters.map(center => `
    <option class="option" value="${center.reference}"
      ${prefilledValues.center === center.reference || prefilledValues.center === center.name ? 'selected' : ''}>
      ${center.name}
    </option>
  `).join('');

  return `
    <select name="center" disabled required>
      <option class="default-option" value="" ${!prefilledValues.center ? 'selected' : ''} style="color: #9ca3af;">
        ${isEnglish ? 'Select a center' : 'Sélectionnez un centre'}
      </option>
      ${centerOptions}
    </select>
  `;
}
```

```
function createServiceSelects(prefilledValues, isEnglish) {
  const typeSelect = createTypeSelect(prefilledValues, isEnglish);
  const commitmentSelect = createCommitmentSelect(prefilledValues, isEnglish);
  return `${typeSelect}${commitmentSelect}`;
}

function createTypeSelect(prefilledValues, isEnglish) {
  const typeOptions = [
    { value: "SERVICE_TYPE_FULL_DESKTOP", label: isEnglish ? 'Private Office' : 'Bureau privé' },
    { value: "SERVICE_TYPE_MEETING", label: isEnglish ? 'Meeting Room' : 'Salle de réunion' },
    { value: "SERVICE_TYPE_FULL_COWORKING", label: 'Coworking' },
    { value: "SERVICE_TYPE_DOMICILIATION", label: 'Domiciliation' }
  ];

  const options = typeOptions.map(opt => `
    <option class="option" value="${opt.value}" ${prefilledValues.type === opt.value ? 'selected' : ''}>
      ${opt.label}
    </option>
  `).join('');

  return `
    <select name="type" disabled required>
      <option class="default-option" value="" ${!prefilledValues.type ? 'selected' : ''} style="color: #9ca3af;">
        ${isEnglish ? 'Select a type' : 'Sélectionnez un type'}
      </option>
      ${options}
    </select>
  `;
}

function createCommitmentSelect(prefilledValues, isEnglish) {
  const commitmentOptions = [
    { value: "short", label: isEnglish ? 'Less than 12 months' : 'Moins de 12 mois' },
    { value: "medium", label: `12 ${isEnglish ? 'months' : 'mois'}` },
    { value: "long", label: isEnglish ? 'More than 12 months' : 'Plus de 12 mois' }
  ];

  const options = commitmentOptions.map(opt => `
    <option class="option" value="${opt.value}" ${prefilledValues.commitment === opt.value ? 'selected' : ''}>
      ${opt.label}
    </option>
  `).join('');

  return `
    <select name="commitment" disabled required>
      <option class="default-option" value="" ${!prefilledValues.commitment ? 'selected' : ''} style="color: #9ca3af;">
        ${isEnglish ? 'Duration' : 'Durée'}
      </option>
      ${options}
    </select>
  `;
}

```

```
function createContactInputs(prefilledValues, isEnglish) {
  return `
    <input type="text" placeholder="${isEnglish ? 'Company name' : 'Nom de l'entreprise'}"
      name="company" value="${prefilledValues.company || ''}" disabled>
    <input type="text" placeholder="${isEnglish ? 'Last name' : 'Nom de famille'}"
      name="lastname" value="${prefilledValues.lastname || ''}" disabled>
    <div style="display: flex; gap: 0.5rem; align-items: center;">
      <input type="email" placeholder="E-mail" name="email"
        value="${prefilledValues.email || ''}" disabled style="flex: 1;">
      <input type="tel" placeholder="${isEnglish ? 'Phone' : 'Téléphone'}" name="phone"
        value="${prefilledValues.phone || ''}" disabled style="flex: 1;">
    </div>
  `;
}

function createFormButtons(formId, isEnglish) {
  return `
    <button class="cancel-btn" onclick="disableForm('${formId}')" disabled>
      ${isEnglish ? 'Cancel' : 'Annuler'}
    </button>
    <button class="submit-btn" onclick="submitForm('${formId}')" disabled>
      ${isEnglish ? 'Send' : 'Envoyer'}
    </button>
  `;
}
```

Annexe 4 : Mail de reporting:

 Rapport Hebdomadaire Chatbot - 11/08/2025 - 17/08/2025

 rapport_hebdomadaire_2025...
27 Ko

Rapport Hebdomadaire Chatbot Acquisition

Période: 11/08/2025 - 17/08/2025

Résumé Exécutif

66

Sessions totales

11

Leads générés

16.67%

Taux de conversion

Répartition Quotidienne

Jour	Sessions	Leads
Monday 11/08	11	3
Tuesday 12/08	11	1
Wednesday 13/08	11	0